大语言模型驱动的优化建模：多智能体协作、形式化验证与求解器信息强化学习深度综述与复现指南

在当前的运筹学（Operations Research, OR）与人工智能（AI）交叉领域，利用大语言模型（LLM）自动化构建数学优化模型已成为研究的前沿热点。传统的优化建模过程高度依赖领域专家的经验，需要将模糊的自然语言需求转化为严谨的数学形式，如线性规划（LP）或混合整数线性规划（MILP），并编写特定的求解器代码（如 Gurobipy 或 Pyomo）[1]。然而，LLM 在处理此类复杂逻辑任务时经常面临幻觉（Hallucinations）、约束遗漏和语法错误等挑战[1]。为了解决这些问题，学术界涌现出一系列基于多智能体协作、形式化中间表示和强化学习（RL）的创新方法。本报告旨在深度梳理这些核心工作，并为“求解器信息强化学习”（Solver-Informed RL, SIRL）的复现提供详尽的技术指导。

多智能体协作与代理式工作流的演进

多智能体系统（Multi-Agent Systems, MAS）通过将复杂的建模任务解耦为多个子任务，由具有不同角色和专业知识的智能体协作完成，显著提升了建模的准确性与鲁棒性[3]。

专家链与任务解耦机制

专家链（Chain-of-Experts, CoE）框架是解决复杂运筹学问题的典型代表。该方法模拟了人类专家的工作流，将问题分解为问题分析、变量定义、目标函数构建和约束条件识别等多个环节[3]。通过引入“建模专家”负责逻辑表述和“求解专家”负责代码实现，系统能够有效地将非结构化文本转化为结构化模型[4]。

在代理式工作流中，OptiMUS 框架展示了求解器反馈在闭环系统中的核心价值。当生成的代码在执行过程中报错时，智能体会捕获求解器日志，并启动自我修复（Self-Correction）机制进行诊断和修正[1]。这种基于错误反馈的迭代过程，使得模型在处理大规模工业问题时具有更高的成功率[1]。

结构化验证与行为一致性

为了确保生成的模型不仅在语法上正确，而且在行为逻辑上与人类预期一致，ReLoop 框架引入了结构化建模与行为验证机制[6]。它通过验证模型在不同参数实例下的表现，确保建模逻辑的通用性[1]。此外，OptiRepair 针对供应链优化等特定领域，提供了基于智能体的闭环诊断，能够识别物理含义上的建模错误，而不仅仅是语法层面的调试[6]。

框架名称	核心特征	适用场景	关键组件

Chain-of-Experts (CoE)	角色扮演与思维解耦	复杂运筹学问题建模	建模专家、求解专家 [3]
OptiMUS	求解器反馈闭环修复	大规模 (MI)LP 问题	求解器日志解析器 [1]
OptiRepair	领域特定诊断	供应链优化模型维护	闭环诊断智能体 [6]
ReLoop	行为验证与逻辑一致性	高可靠性优化建模	结构化验证器 [6]
AlphaOPT	自我提升经验库	跨任务知识迁移	经验库 (RAG) [9]

形式化中间表示与标准化建模

在 LLM 驱动的优化建模中，自然语言的模糊性与求解器代码的严谨性之间存在巨大的语义鸿沟。引入规范化的中间表示（Canonical Intermediate Representation, CIR）是解决这一矛盾的关键手段 [1]。

桥接灵活性与标准化

CIR 旨在从数学底层逻辑出发，定义一套与特定编程语言无关的通用表示法。这种表示法通常包含集合（Sets）、参数（Parameters）、变量（Variables）、目标函数（Objective）和约束（Constraints）五个核心要素 [8]。通过将自然语言描述先转化为 CIR，再由 CIR 生成 Gurobi 或 CPLEX 代码，可以有效地减少模型生成的随机性，提升跨求解器的泛化能力 [8]。

在处理大规模优化问题时，这种“先建模、后代码”的策略尤为重要。它允许研究者在 CIR 层面进行逻辑检查，而无需处理冗长的 Python 脚本。ORGEval 等工具进一步利用图论（Graph-Theoretic）方法，将 CIR 转化为计算图，通过比较模型图结构的相似性来评估 LLM 的建模能力，这比传统的二进制正误评估更具洞察力 [7]。

MILP 的基础模型与代码检索

随着基础模型（Foundation Models）在各领域的普及，针对混合整数线性规划（MILP）的专用模型也开始出现。这些模型在预训练阶段就接触了大量的 .lp 和 .mps 文件，从而获得了更强的数学结构感知能力 [1]。为了进一步辅助生成，基于代码检索（Code Retrieval）的方法（如针对 MILP 实例的代码检索）允许模型参考相似问题的历史代码，通过检索增强生成（RAG）技术减少从零构建模型的难度 [1]。

自动化数据合成与基准测试

高质量的数据是训练高性能建模模型的基石。由于人工标注优化问题的成本极高，研究者开发了

一系列自动化数据合成框架[11]。

OptMATH 与双向合成流水线

OptMATH 是一个可扩展的双向数据合成框架，它通过从数学公式（MF）出发，反向生成自然语言描述（NL），确保了数据对的严谨性[11]。其工作流程包括：

1. 种子生成：利用 53 个种子生成器涵盖物流、供应链、制造等 10 多个真实领域[11]。
2. 反向翻译：将数学模型转化为详细的业务场景描述[13]。
3. 验证与过滤：采用正向建模和拒绝采样（Rejection Sampling），只有当 LLM 能够从 NL 重新构建出原始 MF 时，该数据才会被接受[11]。

Step-Opt 与复杂度进化

Step-Opt 框架通过迭代的问题生成，系统地增加问题的复杂度[17]。它引入了“复杂度进化”（Complexity-Evolving）和“范围进化”（Scope-Evolving）机制，通过分步骤的验证，防止错误在合成过程中传播，从而生成了如 Step-Opt-Instruct 等高质量指令微调数据集[18]。

数据集名称	实例数量	涵盖类型	复杂度评价
NL4OPT	230 - 289	简单线性规划 (LP)	低[3]
Mamo EasyLP	652	线性规划	中低[3]
OptiBench	605	线性、非线性、表格数据	中[7]
OptMATH-Train	201,071	LP, MILP, IP, NLP, SOCP	高[11]
IndustryOR	100	工业级大规模优化	极高[3]

求解器信息强化学习（SIRL）深度解析

报告的核心任务是分析并指导复现《Solver-Informed RL: Grounding Large Language Models for Authentic Optimization Modeling》（arXiv:2505.11792）[1]。该工作首次将可验证奖励强化学习（RLVR）应用于优化建模，标志着从单纯的文本生成向真实物理求解能力的跨越[1]。

核心动机：解决幻觉与接地问题

尽管通过监督微调（SFT）可以使模型模仿建模代码的风格，但在面对从未见过的约束或复杂的系数关系时，LLM 仍会产生幻觉[1]。SIRL 的核心思想是：利用优化求解器（如 Gurobi, COPT, SCIP）作

为客观的“判官”，为模型的输出提供直接、准确的奖励信号[1]。

关键技术一：基于求解器反馈的奖励函数（RLVR）

SIRL 并不依赖于主观的人类偏好，而是通过执行生成的代码来获取标量奖励[12]。

1. 代码执行与结果验证：生成的代码在沙盒环境中运行。如果代码因语法错误无法运行，则给予负奖励[26]。
2. 可行性与目标值：求解器返回模型的可行性状态（Feasible/Infeasible）。如果可行，将得到的目标函数值 y 与地面真值 y^* 进行对比[26]。
3. 细粒度过程奖励（**.lp** 文件统计）：求解器生成的 .lp 文件包含了模型的显式实例表示。SIRL 从中提取变量数量（N_{bin}, N_{int}）、约束类型和优化方向等统计信息[4]。通过将这些结构化特征与真值进行对比，即便最终数值不完全一致，也能为模型提供有价值的梯度引导，缓解了奖励稀疏性问题[2]。

关键技术二：多角色一致性与公式 3 (Equation 3)

在合成训练数据或生成伪标签时，SIRL 采用了一种实例增强的自我一致性方法。系统分配 10 个不同的角色生成候选解答[26]。通过加权投票机制，计算各角色输出特征的共识程度[26]。公式 3 定义了响应 r 的得分 $S(r)$：

$$S(r) = \sum_{f \in F} w_f \cdot \psi_f(r)$$

其中 ψ_f 是特征 f（如目标值、优化方向等）的共识计数，即有多少个角色生成了相同的特征值[26]。这种机制确保了在没有标签的情况下，系统依然能识别出最可靠的推理路径[26]。

关键技术三：部分 KL 散度机制（Partial KL Divergence）

传统的强化学习为了维持模型稳定性，会在所有 token 上施加 KL 惩罚，这往往会抑制模型的创造性推理[4]。SIRL 提出了一种新颖的分阶段控制策略：

- 自由探索阶段（推理与分析）：在模型进行逻辑分析（Chain-of-Thought, CoT）的早期步骤，移除 KL 惩罚。这允许模型扮演“资深专家”，深入探讨多种解题路径，而不受基座模型分布的束缚[4]。
- 严格执行阶段（建模与编码）：当模型进入数学公式描述和求解器代码生成环节时，重新引入 KL 惩罚。这类似于给工程师施加严格的规范手册，确保输出符合严谨的语法和格式要求，防止模型因过度探索而导致崩溃[4]。

SIRL 论文复现指南

复现 SIRL 是一个涉及大模型全流程训练（SFT -> RL）的系统工程。以下是针对导师要求的复现任务的专业指导。

复现内容与技术栈

复现的核心在于搭建一个能够与求解器交互的强化学习环境 [30]。

- 模型选择：建议使用 Qwen2.5-3B 或 1.5B。Qwen 系列在数学和代码任务上表现优异，且 3B 规模在 Google Colab 或 AutoDL 的单张显卡上即可跑通 RL 流程 [11]。
- 强化学习框架：推荐使用 verl (Volcano Engine Reinforcement Learning)。它是目前最灵活、高效的开源 RL 库之一，原生支持 GRPO、PPO 等算法，并且非常易于集成自定义的“求解器奖励函数” [30]。
- 求解器环境：必须在容器或环境中安装 gurobipy。如果没有 Gurobi 商业许可，可以使用开源的 pulp 结合 cbc 求解器，或者安装 Pyomo 并调用 glpk [4]。

复现工作量预估

阶段	任务描述	预计耗时
环境搭建	安装 verl, Ray, gurobipy, Pyomo 等	1-2 天
数据预处理	采样 OptMATH 或 StepOPT 数据，转化为 JSONL 格式	1 天
SFT 预训练	在采样数据上进行基础指令微调，使模型学会基本格式	2-3 天
RLVR 开发	编写 Python 脚本调用求解器，实现奖励计算逻辑	3-5 天
RL 训练实验	在 Colab/AutoDL 上运行强化学习任务，记录指标	5-7 天

评测与分析	在 NL4OPT 等 Benchmark 上进行测试并绘图	2-3 天

训练流程详解

1. 监督微调 (SFT) 阶段

由于基座模型可能不熟悉 SIRL 特有的思维链（CoT）格式，首先需要使用几千条高质量数据进行 SFT [29]。输出应严格遵循三段式结构：<think>（推理逻辑）、<math>（数学模型）、<code>（求解器代码）[4]。

2. 强化学习 (RLVR) 阶段

这是复现的最关键部分。在 verl 框架中，你需要实现一个 RewardModelWorker [30]。

- 输入：模型的生成结果。
- 操作：
  1. 使用正则提取 <code> 块。
  2. 将代码写入临时文件并使用 subprocess 执行。
  3. 解析输出（控制台或生成的 .lp 文件）。
- 奖励逻辑建议：
  - 成功提取代码并运行成功：+0.2
  - 模型可行（Found solution）：+0.3
  - 目标值与真值匹配（误差内）：+1.0
  - 如果报错或语法错误：-0.5 或 0 [27]。

3. 监控指标

实验过程中必须记录以下曲线，这不仅是任务要求，也是判断 RL 是否收敛的关键 [31]：

- 奖励函数变化曲线 **(Reward Curve)**：随着训练进行，平均奖励应稳步上升。
- 熵变化曲线 **(Entropy Curve)**：反映了模型输出的确定性。初期熵值较高（探索），后期应逐渐下降（收敛）[4]。
- **KL** 散度：监控模型相对于 SFT 基准的偏移。
- 成功率 **(Pass@1)**：在测试集上的建模准确率。

数据集采样建议

由于完整的 OptMATH-Train 201k 数据量巨大，复现时建议采样：

- **OptMATH-Train:** 抽取 5000-10000 条涵盖不同类型的题目（LP, MILP）[11]。
- **StepOPT:** 抽取 2000 条复杂题目，测试模型的逻辑极限 [18]。
- 测试集：准备好 NL4OPT (230条), IndustryOR (100条) 进行最终评测 [7]。

深度洞察：运筹学与大模型的融合趋势

通过对上述论文的梳理，可以发现几个显著的二阶趋势：

1. 从“文本生成”向“程序合成”的范式转移：传统的 LLM 评估关注文本流畅度，但在 OR 领域，唯一标准是“可解性”和“正确性” [25]。SIRL 证明了通过将求解器执行引入训练循环，LLM 可以习得超越统计关联的逻辑推理能力 [24]。
2. 中间表示的必要性：随着问题规模的扩大，直接生成代码的难度呈指数级增长。未来，能够生成高质量 CIR 的模型将比直接生成 Python 的模型具有更强的跨行业泛化能力 [8]。
3. 计算资源与推理效率：强化学习虽然训练成本高，但在推理阶段（Test-Time），经过 RL 对齐的模型在 Pass@1 上的表现往往远超基于 Prompting 的大模型 [4]。这对于实时性要求高的生产调度场景至关重要。
4. 形式化验证的回归：过去 OR 专家手动验证模型，现在 EquivaMap 和 ORGEval 实现了自动验证 [7]。这种自动化的“监督者”将使得模型能够在大规模未标注数据上进行自我博弈和进化。

结论与行动建议

大语言模型与运筹学优化的结合正处于爆发前夜。多智能体协作解决了任务分解问题，数据合成解决了训练资源短缺问题，而以 SIRL 为代表的强化学习方法则解决了最核心的“真伪校验”问题 [1]。

对于研究者而言，复现 SIRL 论文不仅仅是跑通代码，更是理解“如何将外部工具的反馈转化为神经网络权重”的过程。建议在复现过程中：

- 优先保证代码解析和执行逻辑的健壮性。
- 细致调节 KL 惩罚系数，观察 Partial KL 对思维链深度的影响。
- 充分利用 verl 提供的监控工具，实时跟踪奖励分布。

通过这一系统的研究与实验，将能够掌握新一代“具备逻辑接地的推理模型”的核心开发技术，为未来更复杂的智能决策系统奠定坚实基础 [5]。

Works cited

1. Solver-Informed RL: Grounding Large Language Models for Authentic Optimization Modeling - arXiv, accessed April 18, 2026, https://arxiv.org/html/2505.11792v2
2. Solver-Informed RL: Grounding Large Language Models for Authentic Optimization Modeling - arXiv, accessed April 18, 2026, https://arxiv.org/pdf/2505.11792
3. LLM for Optimization Modeling - GitHub Pages, accessed April 18, 2026, https://llm4or.github.io/LLM4OR/
4. Solver-Informed RL: grounding LLMs for authentic optimization modeling - Medium, accessed April 18, 2026,

https://medium.com/@oneday88/solver-informed-rl-grounding-llms-for-authentic-optimization-modeling-2a761c03cbc4
5. The AI Engineer's Secret Weapon: A Deep Dive into the Sequential Thinking LP Solver, accessed April 18, 2026, https://skywork.ai/skypage/en/ai-engineer-sequential-thinking-solver/1978352817731399680
6. ReLoop: Structured Modeling and Behavioral Verification for Reliable LLM-Based Optimization - arXiv, accessed April 18, 2026, https://arxiv.org/pdf/2602.15983
7. antgroup/LLMOPT: The LLMOPT project offers a ... - GitHub, accessed April 18, 2026, https://github.com/caigaojiang/LLMOPT
8. LLMOPT: Learning to Define and Solve General Optimization Problems from Scratch - arXiv, accessed April 18, 2026, https://arxiv.org/html/2410.13213v2
9. AlphaOPT: Formulating Optimization Programs with Self-Improving LLM Experience Library, accessed April 18, 2026, https://arxiv.org/html/2510.18428v1
10. Cardinal-Operations/OPTEngine: OPT-Engine: Benchmarking LLMs on Optimization Modeling Across Complexity Scales - GitHub, accessed April 18, 2026, https://github.com/Cardinal-Operations/OPTEngine
11. optsuite/OptMATH · GitHub, accessed April 18, 2026, https://github.com/optsuite/OptMATH
12. Solver-Informed RL: Grounding Large Language Models for Authentic Optimization Modeling - arXiv, accessed April 18, 2026, https://arxiv.org/html/2505.11792v3
13. [2502.11102] OptMATH: A Scalable Bidirectional Data Synthesis Framework for Optimization Modeling - arXiv, accessed April 18, 2026, https://arxiv.org/abs/2502.11102
14. OptMATH - A Scalable Bidirectional Data Synthesis Framework, accessed April 18, 2026, https://auroralhl.github.io/assets/projects/optmath/
15. OptMATH: A Scalable Bidirectional Data Synthesis Framework for Optimization Modeling, accessed April 18, 2026, https://openreview.net/forum?id=9P5e6iE4WK
16. Aurora-Gem/OptMATH-Train · Datasets at Hugging Face, accessed April 18, 2026, https://huggingface.co/datasets/Aurora-Gem/OptMATH-Train
17. Step-Opt: Boosting Optimization Modeling in LLMs through Iterative Data Synthesis and Structured Validation - ResearchGate, accessed April 18, 2026, https://www.researchgate.net/publication/392941851_Step-Opt_Boosting_Optimization_Modeling_in_LLMs_through_Iterative_Data_Synthesis_and_Structured_Validation
18. Training LLMs for Optimization Modeling via Iterative Data Synthesis and Structured Validation - ACL Anthology, accessed April 18, 2026, https://aclanthology.org/2025.findings-emnlp.691.pdf
19. Training LLMs for Optimization Modeling via Iterative Data Synthesis and Structured Validation - ACL Anthology, accessed April 18, 2026, https://aclanthology.org/2025.findings-emnlp.691/
20. [2506.17637] Step-Opt: Boosting Optimization Modeling in LLMs through Iterative Data Synthesis and Structured Validation - arXiv, accessed April 18, 2026, https://arxiv.org/abs/2506.17637

21. OptiBench Meets ReSocratic: Measure and Improve LLMs for Optimization Modeling, accessed April 18, 2026, https://iclr.cc/media/iclr-2025/Slides/28850.pdf
22. OptiBench Meets ReSocratic: Measure and Improve LLMs for Optimization Modeling - arXiv, accessed April 18, 2026, https://arxiv.org/abs/2407.09887
23. [2505.11792] Solver-Informed RL: Grounding Large Language Models for Authentic Optimization Modeling - arXiv, accessed April 18, 2026, https://arxiv.org/abs/2505.11792
24. Solver-Informed RL: Grounding Large Language Models for Authentic Optimization Modeling - arXiv, accessed April 18, 2026, https://arxiv.org/pdf/2505.11792?
25. Solver-Informed RL: Grounding Large Language Models for Authentic Optimization Modeling - Hugging Face, accessed April 18, 2026, https://huggingface.co/papers/2505.11792
26. NeurIPS Poster Solver-Informed RL: Grounding Large Language Models for Authentic Optimization Modeling, accessed April 18, 2026, https://neurips.cc/virtual/2025/poster/119660
27. Execution-Verified Reinforcement Learning for Optimization Modeling - arXiv, accessed April 18, 2026, https://arxiv.org/pdf/2604.00442
28. Solver-Informed RL: Grounding Large Language Models for Authentic Optimization Modeling - OpenReview, accessed April 18, 2026, https://openreview.net/pdf?id=80L235oVBe
29. OR-R1: Automating Modeling and Solving of Operations Research Optimization Problem via Test-Time Reinforcement Learning - ResearchGate, accessed April 18, 2026, https://www.researchgate.net/publication/397556040_OR-R1_Automating_Modeling_and_Solving_of_Operations_Research_Optimization_Problem_via_Test-Time_Reinforcement_Learning
30. Train CodeFu-7B with veRL and Ray on Amazon SageMaker Training jobs - AWS, accessed April 18, 2026, https://aws.amazon.com/blogs/machine-learning/train-codefu-7b-with-verl-and-ray-on-amazon-sagemaker-training-jobs/
31. Tutorial: Scaling Reinforcement Learning with verl on GKE - Google Developer forums, accessed April 18, 2026, https://discuss.google.dev/t/tutorial-scaling-reinforcement-learning-with-verl-on-gke/336370
32. verl: Volcano Engine Reinforcement Learning for LLMs - GitHub, accessed April 18, 2026, https://github.com/verl-project/verl
33. Welcome to verl's documentation! — verl documentation, accessed April 18, 2026, https://verl.readthedocs.io/
34. CALM Before the STORM: Unlocking Native Reasoning for Optimization Modeling - arXiv, accessed April 18, 2026, https://arxiv.org/pdf/2510.04204
35. NeurIPS '25 Paper Finder, accessed April 18, 2026, https://neurips.aipapertrails.com/
36. VerlTool: Towards Holistic Agentic Reinforcement Learning with Tool Use - arXiv, accessed April 18, 2026, https://arxiv.org/html/2509.01055v1

37. OPT-Engine: Benchmarking the Limits of LLMs in Optimization Modeling via Complexity Scaling - arXiv, accessed April 18, 2026, https://arxiv.org/html/2601.19924v1